

Training Guide — Creating the Procedural Starting Point

Purpose: Build the `PLC_NoOOP` project from scratch — the deliberate "before" state that the OOP training refactors step by step

Platform: TwinCAT 3 Structured Text (IEC 61131-3)

Companion guide: `training-procedural-to-oop.md`

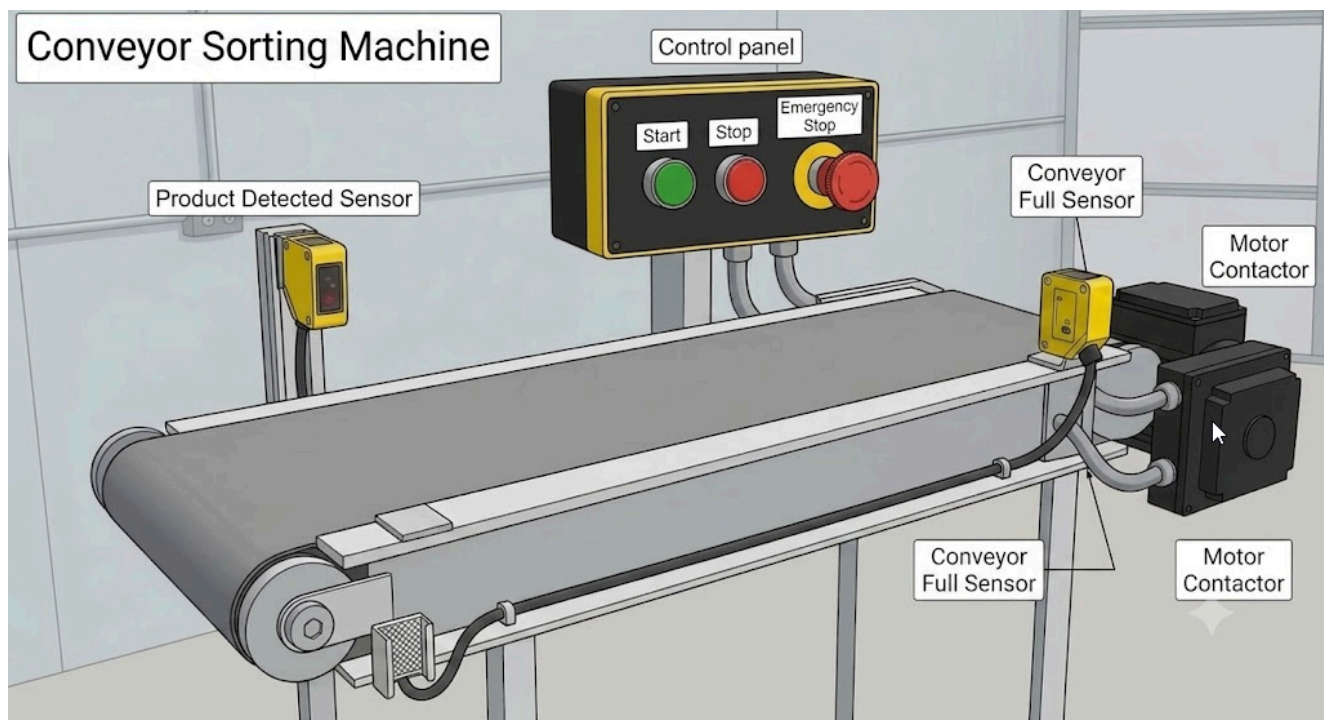
How to Use This Guide

The OOP training (`training-procedural-to-oop.md`) starts with a working procedural program already in place. This guide creates that program.

Work through each step in order. When you are done, you will have a functional TwinCAT 3 solution —

`PLC_NoOOP` — that controls a conveyor sorting machine using flat global variables, a single MAIN program, and no OOP concepts. It will work correctly. It will also have a set of deliberate pain points that you will feel more clearly once you have built it yourself.

The Machine



The machine you are programming is a conveyor sorting machine.

Control panel (top of machine):

- Green button — **Start** (normally open: circuit open at rest, closes when pressed)
- Red button — **Stop** (normally closed: circuit closed at rest, opens when pressed)
- Yellow mushroom button — **Emergency Stop** (normally closed: same behaviour as Stop but latching)

Conveyor sensors:

- **Product Detected Sensor** — mounted above the belt. Normally closed. Goes FALSE when a product passes under it.
- **Conveyor Full Sensor** — mounted at the downstream end. Normally closed. Goes FALSE when the destination is full.

Actuator:

- **Motor Contactor** — a digital output. TRUE = motor running, FALSE = motor stopped.

Behaviour:

1. Press Start while Stop is released → enter run mode
 2. Press Stop or E-Stop → leave run mode immediately
 3. In run mode: if a product is detected, keep the motor running for at least 5 seconds after the last product (off-delay timer)
 4. If the destination conveyor is full, inhibit the motor even in run mode
-

Prerequisites

- TwinCAT 3 XAE installed (tested on 4024.x)
 - A TwinCAT-compatible target (local runtime, EtherCAT coupler, or simulation mode)
 - Basic familiarity with opening TwinCAT XAE and navigating the Solution Explorer
-

Step 1 — Create the TwinCAT Solution

1. Open **TwinCAT XAE** (the Visual Studio shell with TwinCAT integration).
2. Go to **File → New → Project...**
3. In the New Project dialog:
 - Filter or browse to **TwinCAT** in the left panel.
 - Select **TwinCAT XAE Project**.
 - Set **Name** to **OOP** (this is the solution/system name; the PLC project inside it will be named separately).
 - Choose your preferred save location.
 - Click **OK**.

TwinCAT creates a solution with a system node (**OOP**) in the Solution Explorer. You will see **SYSTEM**, **MOTION**, **PLC**, **SAFETY**, **C++**, and **I/O** nodes.

Step 2 — Add the PLC Project

1. Right-click the **PLC** node in Solution Explorer → **Add New Item...**
2. Select **Standard PLC Project**.
3. Set **Name** to **PLC_NoOOP**.

4. Click **Open**.

TwinCAT creates a PLC project with the following structure:

```
PLC_No00P
├─ External Types
├─ References
├─ DUTs
├─ GVLs
├─ POU
├─ MAIN (PRG)
├─ VISUs
└─ PlcTask (Task configuration)
```

You will work inside **GVLs** and **POUs**.

Step 3 — Create the Global Variable List (GVL)

The GVL holds all hardware-mapped I/O variables. In a procedural program, this is the single flat list of every terminal the program touches.

3.1 Add the GVL file

1. Right-click **GVLs** → **Add** → **Global Variable List...**
2. Set **Name** to **GVL**.
3. Click **Open**.

A new file **GVL.TcGVL** opens in the editor.

3.2 Enter the variable declarations

Replace the entire content of the declaration editor with the following:

```
{attribute 'qualified_only'}
VAR_GLOBAL
    EmergencyStopButton    AT%I* : BOOL;
    StopButton             AT%I* : BOOL;
    StartButton            AT%I* : BOOL;
    ProductDetectedSensor  AT%I* : BOOL;
    ConveyorFullSensor     AT%I* : BOOL;

    MotorContactor         AT%Q* : BOOL;
END_VAR
```

3.3 What each declaration means

```
{attribute 'qualified_only'}
```

This pragma forces every reference to use the GVL name as a prefix: **GVL.StartButton**, not just **StartButton**. This prevents accidental name collisions and makes it immediately clear in MAIN that a variable is a hardware-mapped global.

* **AT%I** *

The **AT** keyword binds the variable to a physical memory address. **%I** means "input, auto-assign address". TwinCAT reserves a slot in its I/O memory and lets you link it to a specific terminal channel later in the I/O mapping step. The means the address is not hardcoded yet.

AT%Q

Same as above but for outputs (**%Q** = output). **MotorContactor** is the only output on this machine.

Variable naming

The names follow PascalCase with full descriptive names. This is the conventional style for procedural PLC code. You will see later how the OOP version uses camelCase and drops the type hints — but for now, this is readable and straightforward.

3.4 Save the file

Press Ctrl+S or use File → Save.

Step 4 — Write the MAIN Program

Open POU's → MAIN in Solution Explorer. The MAIN program already exists (TwinCAT creates it by default). You will write into both sections: the declaration area (top) and the implementation area (bottom).

4.1 Declaration section

The declaration section at the top of MAIN defines local variables — variables that exist only within this program and persist their values between scan cycles.

Replace the declaration section content with:

```
PROGRAM MAIN
VAR
    ConveyorControl : BOOL;
    Timeout          : TOF;
END_VAR
```

ConveyorControl : BOOL

A flag that represents whether the machine is in run mode. It is set to TRUE when Start is pressed and Stop/E-Stop are not active. It is the internal "machine running" state.

Timeout : TOF

An instance of the standard **TOF** (Timer Off-Delay) function block. **TOF** starts timing when its **IN** input goes FALSE and holds its **Q** output TRUE for the duration **PT** after that transition. In this machine, **TOF** keeps the motor running for 5 seconds after the last product is detected.

4.2 Implementation section

The implementation section contains the executable logic. Replace the implementation section content with:

```
// Conveyor control is ok when the ES nor the stop button is pushed
IF NOT GVL.StopButton OR NOT GVL.EmergencyStopButton THEN
    ConveyorControl := FALSE;

ELSIF gvl.StartButton AND gvl.StopButton THEN
    ConveyorControl := TRUE;

END_IF

// Minimum time the conveyor keeps on running
Timeout(IN := ConveyorControl AND gvl.ProductDetectedSensor, PT := T#5S, Q =>, ET =>);

// Conveyor keeps on running until not full and some extra time
gvl.MotorContactor := ConveyorControl AND Timeout.Q AND NOT gvl.ConveyorFullSensor;
```

4.3 Step-by-step logic walkthrough

Block 1 — Start/Stop control:

```
IF NOT GVL.StopButton OR NOT GVL.EmergencyStopButton THEN
    ConveyorControl := FALSE;
ELSIF gvl.StartButton AND gvl.StopButton THEN
    ConveyorControl := TRUE;
END_IF
```

The Stop and E-Stop buttons are normally closed (NC). When at rest (not pressed), the circuit is closed and the PLC input reads **TRUE**. When pressed, the circuit opens and the input reads **FALSE**.

This is why the first condition reads **NOT GVL.StopButton** — because **StopButton = FALSE** means it is pressed. The **NOT** inverts this so the condition is **TRUE** when the button is pressed.

NOT GVL.StopButton OR NOT GVL.EmergencyStopButton means: "if either the Stop OR the E-Stop is pressed, go to safe state." This uses short-circuit OR: if Stop is pressed, the E-Stop does not even need to be checked.

The second branch **gvl.StartButton AND gvl.StopButton** means: Start is pressed (normally open, so **TRUE** = pressed) AND Stop is not pressed (normally closed, so **TRUE** = released). This is the only condition that sets **ConveyorControl := TRUE**.

The IF/ELSIF structure gives the stop condition priority: if the first branch triggers, the machine goes to safe state regardless of Start. The ELSIF only evaluates if the first branch was not taken.

Block 2 — Off-delay timer:

```
Timeout(IN := ConveyorControl AND gvl.ProductDetectedSensor, PT := T#5S, Q =>, ET =>);
```

The **TOF** function block is called every scan cycle. Its inputs are:

- **IN** — starts the timer while TRUE; when IN goes FALSE the timer counts down
- **PT** — the preset time (5 seconds)

`Q` and `ET` are outputs written to by the TOF block. The `⇒` syntax means "write this output to..." — here with nothing on the right, the values are computed but not captured in named variables. `Timeout.Q` and `Timeout.ET` are still accessible as members of the `Timeout` instance.

`ConveyorControl AND gvl.ProductDetectedSensor` :

- `ProductDetectedSensor` is normally closed. At rest (no product), input = `TRUE`.
- When a product passes under the sensor, input = `FALSE`.
- So `gvl.ProductDetectedSensor = FALSE` means a product is present.
- `ConveyorControl AND gvl.ProductDetectedSensor` is therefore TRUE when: the machine is in run mode AND no product is currently under the sensor.

Wait — this means the timer's IN goes FALSE when a product is detected, which is when we want to keep the motor running. The TOF off-delay then holds `Q` TRUE for 5 seconds after the product passes (after IN returns to TRUE). This is the intended behaviour: the motor runs for at least 5 seconds after the last product.

Block 3 — Motor output:

```
gvl.MotorContactor := ConveyorControl AND Timeout.Q AND NOT gvl.ConveyorFullSensor;
```

Three conditions must all be TRUE for the motor to run:

1. `ConveyorControl` — machine is in run mode
2. `Timeout.Q` — the timer says we should be running (product detected within the last 5 seconds, or timer still counting)
3. `NOT gvl.ConveyorFullSensor` — destination is not full. `ConveyorFullSensor` is NC, so `TRUE` = not full. `NOT gvl.ConveyorFullSensor = TRUE` means it IS full. Wait — this inverts to: `NOT TRUE = FALSE` when full, which stops the motor. Correct.

Note: `ConveyorFullSensor` being NC means it reads `TRUE` when the destination is NOT full (circuit closed = clear path). `NOT gvl.ConveyorFullSensor` is therefore TRUE only when the sensor reads FALSE, which means the destination IS full — stopping the motor. This is the expected behaviour but the logic requires you to hold the NC wiring convention in your head to follow it.

4.4 Save the file

Press Ctrl+S.

Step 5 — Configure the Task

TwinCAT runs PLC code from a real-time task. The default `PlcTask` created with the project is suitable for training.

1. In Solution Explorer, expand PLC_NoOOP → PlcTask.
2. Double-click PlcTask to open the task settings.

3. Verify the settings:

- Cycle ticks: **1 (with the default 1 ms base time, this gives a 1 ms cycle)**
- Priority: **20 (default)**

4. **Scroll down to the POU's* tab and confirm** MAIN *is listed as the task's cyclic call. It should be added automatically.*

No changes are required here for a basic training setup.

Step 6 — Map the I/O

The AT%I and AT%Q variables reserve slots in TwinCAT's process image but are not yet linked to physical terminals. This step connects each variable to a real (or simulated) I/O channel.

6.1 Build the solution first

Before mapping, TwinCAT needs to compile the PLC project so it knows which variables exist.

1. Press **F11** or go to **Build → Build Solution**.
2. Check the output window for errors. Expect zero errors.

6.2 Open the I/O tree

In Solution Explorer, expand the **I/O** node. You should see a **Devices** node. If you have physical EtherCAT hardware connected, your terminals appear here. If not, you can add a simulated device:

- Right-click **Devices** → **Scan** to detect connected hardware, or
- Right-click **Devices** → **Add New Item** → select a virtual/EtherCAT simulation device for software-only testing.

6.3 Link each GVL variable

For each variable in the GVL, link it to an I/O channel:

1. In Solution Explorer, navigate to **PLC_NoOOP → PLC_NoOOP Instance → GVL**.
2. Each variable appears with its linked address shown in the **Linked to** column (initially empty).
3. Click a variable (e.g. EmergencyStopButton) → in the Properties panel, use the **Linked to** field, or right-click → **Change Link...**
4. In the Link dialog, navigate the I/O tree to the correct terminal channel and click **OK**.

Repeat for all six variables. The mapping table below shows a typical assignment for a physical setup with a Beckhoff EK1100 coupler + digital input/output terminals:

GVL Variable	Type	Typical terminal	Channel type
EmergencyStopButton	AT%I	EL1008 channel 1	Digital Input
StopButton	AT%I	EL1008 channel 2	Digital Input
StartButton	AT%I	EL1008 channel 3	Digital Input

ProductDetectedSensor	AT%I	EL1008 channel 4	Digital Input
ConveyorFullSensor	AT%I	EL1008 channel 5	Digital Input
MotorContactor	AT%Q	EL2008 channel 1	Digital Output

If using simulation mode only: right-click each I/O variable → **Add to Watch** → manually force values in the online watch window during testing.

Step 7 — Activate and Test

7.1 Activate the configuration

1. Press **Ctrl+Shift+F4** or click the **Activate Configuration** button (TwinCAT icon with a green arrow).
2. When prompted "Restart TwinCAT system in Run Mode?" — click **OK**.
3. If prompted to auto-start the PLC — click **Yes**.

7.2 Log in to the PLC

1. In Solution Explorer, right-click **PLC_NoOOP** → **Login** (or press **F11** then **Ctrl+F5**).
2. If prompted to download the program — click **Download**.
3. Press **F5** (Run) to start execution.

7.3 Test the behaviour in sequence

With the program running, use the online watch window or physical hardware to verify each behaviour:

Test 1 — Start the conveyor:

- Ensure Stop = TRUE (NC, released) and E-Stop = TRUE (NC, released)
- Set Start = TRUE (simulate pressing)
- Expect: **ConveyorControl = TRUE**
- Set Start = FALSE (released)
- Expect: **ConveyorControl** remains TRUE (latched by the ELSIF structure only triggering on the next TRUE edge of StartButton)

Note: the current MAIN does not latch — it re-evaluates every scan. **ConveyorControl** stays TRUE as long as StartButton was TRUE at the last scan and StopButton is still TRUE. If StartButton goes FALSE, the ELSIF is FALSE, so ConveyorControl does not change (no branch sets it to FALSE). It stays at its last value. This is the implicit latch behaviour of the IF/ELSIF with no ELSE.

Test 2 — Stop the conveyor:

- With conveyor running, set Stop = FALSE (simulate NC button pressed, circuit opens)
- Expect: **ConveyorControl = FALSE** immediately

Test 3 — Product detection and timer:

- With conveyor running (`ConveyorControl = TRUE`)
- Set `ProductDetectedSensor = FALSE` (simulate product under sensor, NC circuit opens)
- Observe `Timeout.Q = TRUE` — motor runs
- Set `ProductDetectedSensor = TRUE` (product passed)
- Observe `Timeout.ET` counting up from 0 toward 5 seconds
- After 5 seconds: `Timeout.Q = FALSE` → `MotorContactor = FALSE` → motor stops

Test 4 — Conveyor full inhibit:

- With conveyor running and product detected
- Set `ConveyorFullSensor = FALSE` (NC sensor reads full)
- Expect: `MotorContactor = FALSE` even though `ConveyorControl = TRUE` and timer is active

Step 8 — Review the Completed Program

Here is the complete program for reference.

GVL.TcGVL

```
{attribute 'qualified_only'}
VAR_GLOBAL
    EmergencyStopButton    AT%I* : BOOL;
    StopButton             AT%I* : BOOL;
    StartButton            AT%I* : BOOL;
    ProductDetectedSensor  AT%I* : BOOL;
    ConveyorFullSensor     AT%I* : BOOL;

    MotorContactor         AT%Q* : BOOL;
END_VAR
```

MAIN.TcPOU — Declaration

```
PROGRAM MAIN
VAR
    ConveyorControl : BOOL;
    Timeout         : TOF;
END_VAR
```

MAIN.TcPOU — Implementation

```
// Conveyor control is ok when the ES nor the stop button is pushed
IF NOT GVL.StopButton OR NOT GVL.EmergencyStopButton THEN
    ConveyorControl := FALSE;

ELSIF gvl.StartButton AND gvl.StopButton THEN
    ConveyorControl := TRUE;

END_IF

// Minimum time the conveyor keeps on running
Timeout(IN := ConveyorControl AND gvl.ProductDetectedSensor, PT := T#5S, Q =>, ET =>);

// Conveyor keeps on running until not full and some extra time
gvl.MotorContactor := ConveyorControl AND Timeout.Q AND NOT gvl.ConveyorFullSensor;
```

Files created: 2 (`GVL.TcGVL`, `MAIN.TcPOU`)

Lines of logic: 6

Total variables: 8 (6 global I/O + 2 local)

Step 9 — Note the Pain Points

Before moving on to the OOP refactoring, pause and deliberately read through the code once more. Keep these observations in mind — they are the exact problems the OOP training will solve.

Pain point 1 — Hardware knowledge embedded in logic

```
IF NOT GVL.StopButton OR NOT GVL.EmergencyStopButton THEN
```

The `NOT` here is not logic — it is a translation of the NC wiring convention. To understand this line, you must already know that these buttons are normally closed. There is no way to read this line and understand the intent* without that external knowledge. A new engineer reading this code must study the electrical drawings before the code makes sense.

Pain point 2 — The confusing start condition

```
ELSIF gvl.StartButton AND gvl.StopButton THEN
```

The Stop button appears in the Start condition. Why would you check the Stop button when starting? Because the Stop button is NC and must be TRUE (released) for the condition to pass. This is correct logic but reads as though the machine starts when both Start AND Stop are simultaneously pressed. It does not. The normally-closed context inverts the intuition.

Pain point 3 — NC sensor polarity hidden in NOT

```
gvl.MotorContactor := ConveyorControl AND Timeout.Q AND NOT gvl.ConveyorFullSensor;
```

`NOT gvl.ConveyorFullSensor` means "the sensor is reading FALSE", which means the destination IS full. The intent is "stop the motor when the conveyor is full" but the code reads "stop when NOT ConveyorFullSensor". You must hold the NC convention in your head to parse this correctly.

Pain point 4 — The GVL is a wiring list, not a machine description

```
EmergencyStopButton  AT%I* : BOOL;  
StopButton           AT%I* : BOOL;  
StartButton          AT%I* : BOOL;  
...
```

This is a list of electrical terminals, not a description of a machine. There is no way to tell from the GVL alone what type of button each is, what its normal state is, or where it is physically located. All of that knowledge exists only in electrical drawings or the head of whoever wired the panel.

Pain point 5 — Zero reusability

If you need a second conveyor, you copy the entire program, rename every variable, and manage two parallel sets of code. There is no way to share logic between instances because the logic is fused with specific variable names. The word "ConveyorControl" is baked into a single program — it cannot exist twice with different values.

Pain point 6 — Sensor type changes cascade through the program

If the `ConveyorFullSensor` were changed from NC to NO (normally open), you would need to:

1. Find every reference to `gvl.ConveyorFullSensor` in MAIN
2. Decide whether to add or remove `NOT`
3. Verify no other program also references the sensor

In this example there is only one reference. In a real machine with 40 sensors across 10 programs, this becomes a risky edit that touches many files.

What Comes Next

With `PLC_NoOOP` built and working, you are ready for the OOP training guide.

Open `training-procedural-to-oop.md` and begin at **Part 3 — Step-by-Step Refactoring**. That guide takes the exact code you just wrote and transforms it step by step into a fully OOP-structured program. Each step introduces one concept, shows the code change, and explains exactly what problem it solves — many of which you have just experienced firsthand by building this program.

The final OOP program has 14 files instead of 2. Every one of them is under 10 lines of logic. The MAIN program becomes 2 lines. The pain points listed above all disappear.